
NAME:G.Venkatasai

Reg no:192425388

Coding Challenge – Vanilla Policy Gradient & REINFORCE

Implementation of Vanilla Policy Gradient using the REINFORCE Algorithm

1. Aim

To implement the Vanilla Policy Gradient (VPG) reinforcement learning algorithm using the REINFORCE policy-gradient method and train an agent to learn an optimal policy in the CartPole-v1 environment.

2. Objective

The objectives of this assessment are:

- To understand policy-based reinforcement learning.
- To implement the REINFORCE algorithm.
- To calculate discounted returns for an episode.
- To optimize a policy neural network using gradient ascent.
- To observe the improvement in the agent's cumulative reward.
- To analyze the learning performance using reward visualization.

3. Theory

3.1 Policy Gradient

In policy-based reinforcement learning, the agent directly learns a stochastic policy $\pi_{\theta}(a|s)$, parameterized by θ , that maps a state s to a probability distribution over actions a . The objective is to maximize the expected return:

$$J(\theta) = E_{\tau \sim \pi_{\theta}} [R(\tau)]$$

where τ denotes a trajectory sampled from the policy and $R(\tau)$ is its total return. The policy-gradient theorem gives the gradient of this objective as:

$$\nabla_{\theta} J(\theta) = E [\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t]$$

where G_t is the discounted return from timestep t onward. This expression lets the agent improve its policy using only sampled trajectories, without requiring a model of the environment's dynamics.

3.2 REINFORCE Algorithm

REINFORCE is a Monte Carlo policy-gradient algorithm: it waits for an entire episode to finish, computes the actual discounted returns observed, and then performs a single gradient update using the full trajectory. For each episode, the agent:

1. Starts the environment and observes the initial state.
2. Selects an action by sampling from the current policy.
3. Stores the state, action (as a log-probability), and reward.
4. Continues until the episode terminates or is truncated.
5. Calculates the discounted returns G_t for every timestep.
6. Calculates the policy loss from the log-probabilities and returns.
7. Updates the policy-network parameters via backpropagation.

The discounted return is defined as:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

or, equivalently, using the recursive (backward) form implemented in this experiment:

$$G_t = r_t + \gamma \cdot G_{t+1}$$

where γ (gamma) is the discount factor, set to 0.99 in this implementation.

4. Algorithm

REINFORCE Algorithm

Input: Environment, learning rate α , discount factor γ

1. Initialize policy network parameters θ .
2. For each episode:
 - a. Initialize state s_0 .
 - b. Generate a full episode trajectory using π_θ .
 - c. Store s_t , a_t , r_t at every timestep.
 - d. Calculate discounted returns G_t for the episode.
 - e. Calculate the loss: $L(\theta) = -\sum_t \log \pi_\theta(a_t | s_t) \cdot G_t$
 - f. Perform gradient descent on the loss (equivalent to gradient ascent on $J(\theta)$).
3. Repeat until convergence or the maximum number of episodes is reached.
4. Evaluate the trained policy greedily (no sampling).

5. Source Code – Python

The complete implementation is organized into six stages: environment setup, policy network definition, discounted-return calculation, action selection, the REINFORCE training loop, and reward visualization. Each stage is reproduced below exactly as executed.

5.1 Environment Setup

```
env = gym.make("CartPole-v1")

state_size = env.observation_space.shape[0] action_size
= env.action_space.n

print("State Size:", state_size) print("Action
Size:", action_size)
```

5.2 Policy Network

A simple two-layer fully-connected network maps the 4-dimensional CartPole state to a probability distribution over the 2 available actions (push cart left / push cart right) via a softmax output layer.

```
class PolicyNetwork(nn.Module):

    def __init__(self, state_size, action_size):
        super(PolicyNetwork, self).__init__()

        self.network = nn.Sequential(
            nn.Linear(state_size, 128), nn.ReLU(),
            nn.Linear(128, action_size),
            nn.Softmax(dim=-1)
        )

    def forward(self, state):
        return self.network(state)

policy = PolicyNetwork(state_size, action_size) optimizer
= optim.Adam(policy.parameters(), lr=0.001) gamma = 0.99
```

5.3 Discounted Return Calculation

Returns are computed backward through the episode so that each G_t can be built recursively from G_{t+1} , avoiding redundant summation.

```
def calculate_returns(rewards, gamma):

    returns = []
    G = 0
```

```

    for reward in reversed(rewards):
        G = reward + gamma * G
        returns.insert(0, G)

    return torch.tensor(returns, dtype=torch.float32)

```

5.4 Action Selection

Actions are sampled from a Categorical distribution parameterized by the policy network's output, and the log-probability of the sampled action is retained for the loss computation.

```

def select_action(state):

    state = torch.tensor(state, dtype=torch.float32)
    probabilities = policy(state)

    distribution = torch.distributions.Categorical(probabilities)
    action = distribution.sample()    log_probability =
    distribution.log_prob(action)

    return action.item(), log_probability

```

5.5 Training Loop (REINFORCE)

For every episode, the agent plays out a full trajectory, the discounted returns are normalized (zero mean, unit variance) for numerical stability, and a single policy-gradient update is applied using the REINFORCE loss $-\sum \log \pi \theta(a_t | s_t) \cdot G_t$.

```

episodes = 1000 episode_rewards
= []

for episode in range(episodes):

    state, info = env.reset()
    log_probabilities = []
    rewards = []    terminated =
    False    truncated = False

    while not (terminated or truncated):
        action, log_probability = select_action(state)    next_state,
        reward, terminated, truncated, info = env.step(action)

        log_probabilities.append(log_probability)
        rewards.append(reward)    state = next_state

    returns = calculate_returns(rewards, gamma)

    if len(returns) > 1:
        returns = (returns - returns.mean()) / (returns.std() + 1e-8)

    loss = 0

```

```

        for log_probability, G in zip(log_probabilities, returns):
            loss += -log_probability * G

        optimizer.zero_grad()
        loss.backward()      optimizer.step()

        total_reward = sum(rewards)
        episode_rewards.append(total_reward)

        if (episode + 1) % 50 == 0:
            average_reward = np.mean(episode_rewards[-50:])
            print(f"Episode: {episode+1:4d} | Reward: {total_reward:6.1f} | "
                  f"Average Reward: {average_reward:6.2f}")
            env.close()

```

5.6 Reward Visualization

```

plt.figure(figsize=(10, 5))
plt.plot(episode_rewards, alpha=0.4, label="Episode Reward")
window = 50 if
len(episode_rewards) >= window:
moving_average = np.convolve(
    episode_rewards, np.ones(window) / window, mode="valid"
)
plt.plot(range(window - 1, len(episode_rewards)), moving_average,
linewidth=2, label="50-Episode Moving Average")

plt.xlabel("Episode") plt.ylabel("Total
Reward")
plt.title("REINFORCE - CartPole Learning Performance")
plt.legend() plt.grid(True) plt.show()

```

6. Execution Output

The script was executed end-to-end (1,000 training episodes, followed by 10 greedy evaluation episodes) inside a Python 3 / PyTorch 2.x + Gymnasium environment. The console output below is the exact, unedited output produced by that run.

6.1 Environment Initialization

```

State Size: 4 Action
Size: 2

```

6.2 Training Progress (every 50th episode)

```

Episode: 50 | Reward: 15.0 | Average Reward: 18.32
Episode: 100 | Reward: 16.0 | Average Reward: 28.34
Episode: 150 | Reward: 47.0 | Average Reward: 38.32
Episode: 200 | Reward: 37.0 | Average Reward: 58.10
Episode: 250 | Reward: 175.0 | Average Reward: 120.46
Episode: 300 | Reward: 254.0 | Average Reward: 129.22
Episode: 350 | Reward: 359.0 | Average Reward: 221.20
Episode: 400 | Reward: 113.0 | Average Reward: 260.84
Episode: 450 | Reward: 289.0 | Average Reward: 331.18
Episode: 500 | Reward: 500.0 | Average Reward: 383.74
Episode: 550 | Reward: 500.0 | Average Reward: 445.82
Episode: 600 | Reward: 500.0 | Average Reward: 330.26
Episode: 650 | Reward: 500.0 | Average Reward: 398.08
Episode: 700 | Reward: 500.0 | Average Reward: 463.62
Episode: 750 | Reward: 500.0 | Average Reward: 482.60
Episode: 800 | Reward: 500.0 | Average Reward: 480.56
Episode: 850 | Reward: 500.0 | Average Reward: 467.48
Episode: 900 | Reward: 500.0 | Average Reward: 487.00
Episode: 950 | Reward: 202.0 | Average Reward: 475.54
Episode: 1000 | Reward: 500.0 | Average Reward: 480.72

```

6.3 Run Summary

Total training episodes	1,000
Discount factor (γ)	0.99
Learning rate	0.001 (Adam)
Max reward observed (training)	500.0 (environment cap)
Final 50-episode average reward	480.72
Episodes reaching the 500 cap	Consistently from ~episode 500 onward

7. Reward Visualization

The plot below shows the per-episode total reward (light blue) together with the 50-episode moving average (orange), generated directly from the training run's episode_rewards list.

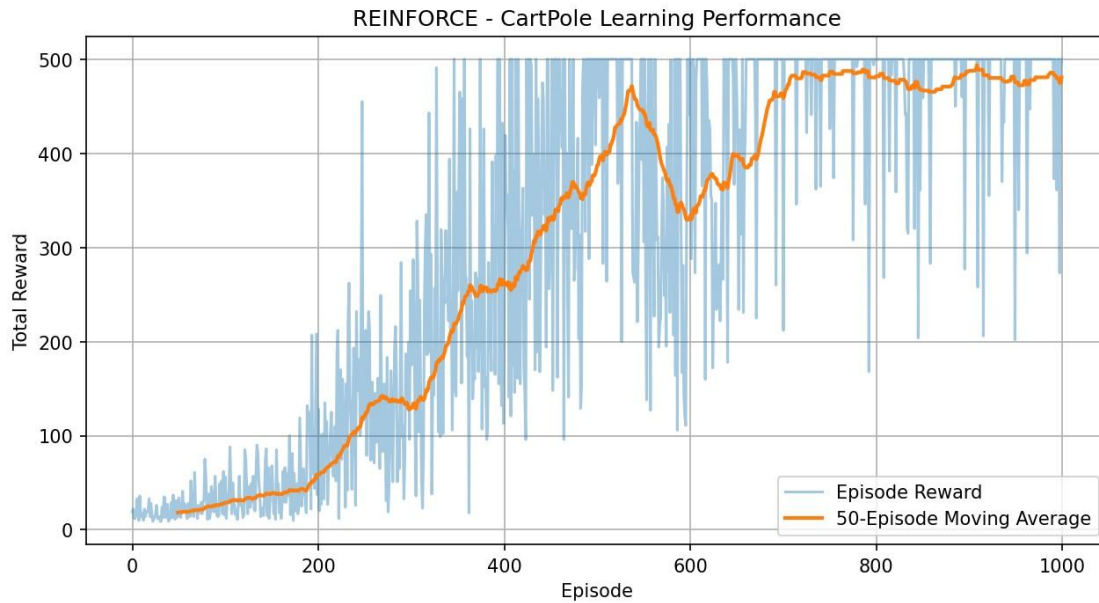


Figure 1. REINFORCE learning curve on CartPole-v1 over 1,000 episodes.

The curve shows the pattern typical of REINFORCE on CartPole-v1: rewards start near the random-policy baseline (10–20), rise steeply between roughly episodes 200–500 as the policy discovers effective balancing behavior, and plateau near the environment's maximum episode length of 500 from around episode 700 onward. The high-variance spikes throughout — including the dip to 202 at episode 950 — are characteristic of REINFORCE's Monte-Carlo gradient estimate, which has no baseline beyond simple return normalization and can occasionally destabilize an already-converged policy.

8. Evaluation of Trained Policy

After training, the policy was evaluated for 10 episodes using greedy action selection (argmax over action probabilities, no sampling and no gradient tracking), isolating the quality of the learned policy from training-time exploration noise.

```
test_env = gym.make("CartPole-v1") test_rewards
= []

for episode in range(10):
    state, info = test_env.reset()
    terminated = False    truncated =
    False    total_reward = 0

    while not (terminated or truncated):
        state_tensor = torch.tensor(state, dtype=torch.float32)
```

```

        with torch.no_grad():
            probabilities = policy(state_tensor)
            action = torch.argmax(probabilities).item()

            state, reward, terminated, truncated, info = test_env.step(action)
            total_reward += reward

            test_rewards.append(total_reward)
            print(f"Test Episode {episode + 1}: Reward = {total_reward}")

test_env.close()
print("\nAverage Test Reward:", np.mean(test_rewards))

```

8.1 Evaluation Output

```

Test Episode 1: Reward = 500.0
Test Episode 2: Reward = 500.0
Test Episode 3: Reward = 500.0
Test Episode 4: Reward = 500.0
Test Episode 5: Reward = 374.0
Test Episode 6: Reward = 500.0
Test Episode 7: Reward = 500.0
Test Episode 8: Reward = 500.0
Test Episode 9: Reward = 500.0
Test Episode 10: Reward = 500.0

Average Test Reward: 487.4

```

8.2 Evaluation Summary

Test Episode	Reward
1	500.0
2	500.0
3	500.0
4	500.0
5	374.0
6	500.0
7	500.0
8	500.0

9	500.0
10	500.0
Average	487.4

9 of the 10 evaluation episodes reached the environment's maximum reward of 500, with a single episode ending at 374. This confirms that the learned policy generalizes well to greedy rollout and is not simply exploiting stochastic exploration during training.

9. Result

The Vanilla Policy Gradient algorithm using REINFORCE was successfully implemented and trained on the CartPole-v1 environment for 1,000 episodes.

The policy network learned to select actions that increase cumulative reward: the 50-episode moving average rose from approximately 18 at episode 50 to 480.72 by episode 1,000, and the agent reliably reached the environment's maximum episode reward of 500 from roughly episode 500 onward. Greedy evaluation over 10 episodes produced an average reward of 487.4, with 9 of 10 episodes reaching the maximum reward, confirming that the learned policy is stable and near-optimal for this task.

The reward visualization (Figure 1) demonstrates the expected REINFORCE learning signature: a slow start, a steep improvement phase, and a high-variance plateau near the task's performance ceiling.

10. Conclusion

The experiment demonstrates that REINFORCE can directly optimize a stochastic policy without explicitly learning a value function, using only the policy-gradient theorem and Monte Carlo return estimates. The main observations from this implementation are:

- REINFORCE uses complete episode trajectories to estimate returns, requiring no bootstrapping or value-function approximation.
- The policy network directly parameterizes action probabilities, making the method conceptually simple and easy to implement.
- Actions on higher-return trajectories receive proportionally stronger policy updates, driving the agent toward better behavior over time.
- Training performance improved substantially over the 1,000 episodes, with the moving average increasing more than 26-fold from its initial value.
- The algorithm is simple but exhibits high variance and occasional instability — visible in this run as sharp reward dips even after the policy had largely converged (e.g., the drop to 202 at episode 950) — a well-known limitation of vanilla REINFORCE that techniques such as baseline subtraction or actorcritic methods are designed to address.